

Report: Indizes

Elisabeth Pusterhofer, 28.09.2026

Streuung

Die Tabelle personen habe ich mit Faker (de_AT, Seed 42) mit 500.000 Datensätzen befüllt. Da Vor- und Nachnamen keine Zahlen sind und keinen Abstand zueinander haben (nominalskaliert), lässt sich keine Varianz direkt berechnen. Deshalb habe ich mit GROUP BY und einer temporären Hilfstabelle (WITH) die absolute Häufigkeit pro Name ermittelt und diese verglichen. Bei 559 verschiedenen Vornamen liegt der Durchschnitt bei ca. 894,5 Vorkommen. Nikola bildet in dieser Tabelle die Ausnahme, da der Name mit doppelter Häufigkeit vorkommt (kommt in weiblicher und männlicher Liste vor). Die Bibliothek Faker verteilt annähernd gleichmäßig.

Index

Ohne Index führt SQLite3 bei WHERE vorname='Anja' einen Full Table Scan durch (ca. 16,7 ms). Mit dem Index, einem B-Baum aus Vornamen und rowid, wird direkt gesucht (ca. 0,45ms). Das ist zwar deutlich schneller, aber braucht dafür mehr Speicherplatz (ohne Index: 11,0 MB; mit Index: 18,2 MB). Es wird für jeden Datensatz ein Indizeintrag gespeichert.

Bias

In der zweiten Datenbank habe ich mit UPDATE personen ... WHERE id % 2 = 0 die Hälfte der Vornamen auf „Sonia“ gesetzt. Die Abfrage nach „Sonia“ mit Index ist langsamer (ca. 100 ms) als ohne (ca. 74 ms). Für jeden der ungefähr 250.000 Treffer wird über die rowid einzeln in der Tabelle gesucht (geringe Selektivität, da sehr viele doppelte Werte enthalten sind). Bei einem Full Table Scan werden die Pages nur einmal sequenziell gelesen. Für seltene Werte wie „Anja“ bleibt der Index schnell. Die Indexgröße bleibt annähernd gleich.

Fazit: Ein Index lohnt sich nur bei hoher Selektivität. Bei Spalten mit sehr häufigen Werten kostet er Speicher, ohne die Abfrage zu beschleunigen.